

PostgreSQL Installation Guide

Chapter 9 | PostgreSQL Complete Guide Difficulty: Beginner | Estimated Reading Time: 45 minutes

Table of Contents

- [Learning Objectives](#)
 - [Theory: Installation Overview](#)
 - [Linux: Ubuntu and Debian](#)
 - [Linux: CentOS and RHEL](#)
 - [macOS Installation](#)
 - [Windows Installation](#)
 - [Docker Installation](#)
 - [Kubernetes Installation](#)
 - [Post-Installation Configuration](#)
 - [Verification Steps](#)
 - [Troubleshooting](#)
 - [ASCII Visual Diagrams](#)
 - [PostgreSQL Commands](#)
 - [SQL Examples](#)
 - [Common Mistakes](#)
 - [Best Practices](#)
 - [Performance Considerations](#)
 - [Interview Questions](#)
 - [Interview Answers](#)
 - [Hands-on Exercises](#)
 - [Solutions](#)
 - [Advanced Notes](#)
 - [Cross-References](#)
-

Learning Objectives

By the end of this chapter, you will be able to:

- Install PostgreSQL on Ubuntu, Debian, CentOS, RHEL, macOS, and Windows
 - Deploy PostgreSQL in Docker and Kubernetes environments
 - Perform post-installation configuration and security hardening
 - Verify a successful installation and connect to the database
 - Troubleshoot the most common installation and connectivity issues
-

Theory: Installation Overview

PostgreSQL can be installed via:

1. **Official PGDG repository** (recommended for Linux) — Always latest version
2. **OS package manager** — Easier but may have older versions
3. **Installer (Windows/macOS)** — GUI wizard from EDB (EnterpriseDB)
4. **Homebrew (macOS)** — Community-maintained, easy updates
5. **Docker** — Containerized, reproducible, portable
6. **Kubernetes** — Production-grade orchestration

7. **Cloud-managed** — AWS RDS, Google Cloud SQL, Azure Database (no installation needed)

8. **Compile from source** — Maximum control, not recommended for beginners

Version Numbering

PostgreSQL uses a `MAJOR.MINOR` scheme (e.g., `16.3`):

- **Major version** (16) changes yearly; contains new features, may require migration
- **Minor version** (.3) is backward-compatible bug/security fixes; always update

The PostgreSQL community supports the last **5 major versions**. Older versions do not receive security updates.

Linux: Ubuntu and Debian

Method 1: Official PGDG Repository (Recommended)

The PostgreSQL Global Development Group (PGDG) maintains official packages for all major distributions. These are always up-to-date and support multiple PostgreSQL versions side-by-side.

```
# Step 1: Install prerequisites
sudo apt-get update
sudo apt-get install -y curl ca-certificates

# Step 2: Add the PostgreSQL APT repository signing key
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc \
  --fail https://www.postgresql.org/media/keys/ACCC4CF8.asc

# Step 3: Add the PGDG repository to sources
sudo sh -c 'echo "deb [signed-by=/usr/share/postgresql-
common/pgdg/apt.postgresql.org.asc] \
  https://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" \
  > /etc/apt/sources.list.d/pgdg.list'

# Step 4: Update and install PostgreSQL (latest version)
sudo apt-get update
sudo apt-get install -y postgresql

# To install a specific version (e.g., 16):
sudo apt-get install -y postgresql-16

# Step 5: Start and enable the service
sudo systemctl start postgresql
sudo systemctl enable postgresql

# Step 6: Verify installation
sudo systemctl status postgresql
psql --version
```

Method 2: Default Ubuntu/Debian Repository (Simpler, May Be Older)

```
sudo apt-get update
sudo apt-get install -y postgresql postgresql-contrib

# Check installed version
psql --version
```

First Login After Installation

```
# Switch to the postgres OS user
sudo -i -u postgres

# Enter PostgreSQL interactive terminal
psql

# OR in one command:
sudo -u postgres psql
```

Ubuntu/Debian Service Management

```
# Start / Stop / Restart / Reload
sudo systemctl start postgresql
sudo systemctl stop postgresql
sudo systemctl restart postgresql
sudo systemctl reload postgresql # Reload config without full restart

# Check status
sudo systemctl status postgresql

# View logs
sudo journalctl -u postgresql -f

# Data directory
ls /var/lib/postgresql/16/main/

# Configuration files
cat /etc/postgresql/16/main/postgresql.conf
cat /etc/postgresql/16/main/pg_hba.conf
```

Linux: CentOS and RHEL

Method 1: Official PGDG Repository (Recommended)

```
# Step 1: Install the PGDG RPM repository
# For RHEL/CentOS 9 (x86_64), PostgreSQL 16:
sudo dnf install -y https://download.postgresql.org/pub/repos/yum/repopms/EL-9-x86_64/pgdg-redhat-repo-latest.noarch.rpm
```

```
# Step 2: Disable the built-in PostgreSQL module (RHEL 8/9)
sudo dnf -qy module disable postgresql

# Step 3: Install PostgreSQL
sudo dnf install -y postgresql16-server postgresql16-contrib

# For RHEL/CentOS 7 (uses yum, not dnf):
# sudo yum install -y https://download.postgresql.org/pub/repos/yum/repos/pms/EL-7-
x86_64/pgdg-redhat-repo-latest.noarch.rpm
# sudo yum install -y postgresql16-server postgresql16-contrib

# Step 4: Initialize the database
sudo /usr/pgsql-16/bin/postgresql-16-setup initdb

# Step 5: Enable and start the service
sudo systemctl enable postgresql-16
sudo systemctl start postgresql-16

# Step 6: Verify
sudo systemctl status postgresql-16
sudo -u postgres psql -c "SELECT version();"

```

RHEL/CentOS Configuration Paths

```
# Data directory
/var/lib/pgsql/16/data/

# Configuration files
/var/lib/pgsql/16/data/postgresql.conf
/var/lib/pgsql/16/data/pg_hba.conf

# Log files
/var/lib/pgsql/16/data/log/

# Binaries
/usr/pgsql-16/bin/

# Service management (same as Ubuntu)
sudo systemctl restart postgresql-16
sudo journalctl -u postgresql-16 -f

```

Firewall Configuration (RHEL)

```
# Allow PostgreSQL through firewall
sudo firewall-cmd --add-service=postgresql --permanent
sudo firewall-cmd --reload

# Or specify port explicitly

```

```
sudo firewall-cmd --add-port=5432/tcp --permanent
sudo firewall-cmd --reload
```

macOS Installation

Method 1: Homebrew (Recommended for Development)

```
# Step 1: Install Homebrew (if not already installed)
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Step 2: Install PostgreSQL
brew install postgresql@16

# Step 3: Add to PATH (add to ~/.zshrc or ~/.bash_profile)
echo 'export PATH="/opt/homebrew/opt/postgresql@16/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

# Step 4: Start PostgreSQL service
brew services start postgresql@16

# To start manually (without auto-start on login):
# /opt/homebrew/opt/postgresql@16/bin/postgres -D /opt/homebrew/var/postgresql@16

# Step 5: Verify
psql --version
psql -U $(whoami) postgres # Connect using your macOS username
```

Homebrew Service Management

```
# Start at login (auto-start)
brew services start postgresql@16

# Stop auto-start
brew services stop postgresql@16

# Restart
brew services restart postgresql@16

# Check status
brew services list | grep postgresql

# Data directory
ls /opt/homebrew/var/postgresql@16/

# Logs
tail -f /opt/homebrew/var/log/postgresql@16.log
```

```
# Update PostgreSQL
brew upgrade postgresql@16
```

Method 2: EDB Installer (GUI)

1. Download from <https://www.enterprisedb.com/downloads/postgres-postgresql-downloads>
2. Run the .dmg installer
3. Follow wizard: choose version, data directory, port (default: 5432), superuser password
4. Optionally install Stack Builder for additional tools

Method 3: Postgres.app (Simplest for macOS)

1. Download from <https://postgresapp.com>
2. Move to Applications folder
3. Click "Initialize" — PostgreSQL starts immediately
4. Add CLI tools to PATH:

```
sudo mkdir -p /etc/paths.d &&
echo /Applications/Postgres.app/Contents/Versions/latest/bin | sudo tee
/etc/paths.d/postgresapp
```

Windows Installation

Method 1: EDB Interactive Installer (Recommended)

1. Download from: <https://www.enterprisedb.com/downloads/postgres-postgresql-downloads>
Choose: Windows x86-64, PostgreSQL 16.x
2. Run the installer as Administrator
3. Installation wizard steps:
 - Installation Directory: C:\Program Files\PostgreSQL\16
 - Data Directory: C:\Program Files\PostgreSQL\16\data
 - Password: Set a strong superuser password (REMEMBER THIS!)
 - Port: 5432 (default)
 - Locale: Default
4. Stack Builder (optional): installs additional tools (pgAdmin, PostGIS, ODBC drivers)
5. After installation, PostgreSQL runs as a Windows service named "postgresql-x64-16"

Windows Service Management

```
# PowerShell (as Administrator):

# Start/Stop/Restart service
Start-Service postgresql-x64-16
Stop-Service postgresql-x64-16
Restart-Service postgresql-x64-16
```

```
# Check service status
Get-Service postgresql-x64-16

# View in Services GUI
services.msc
```

Windows Command Line

```
REM Add PostgreSQL bin to PATH (or add permanently via System Environment Variables)
set PATH=%PATH%;C:\Program Files\PostgreSQL\16\bin

REM Connect to PostgreSQL
psql -U postgres

REM Run a command
psql -U postgres -c "SELECT version();"

```

Windows Firewall

```
# Allow PostgreSQL through Windows Firewall (PowerShell as Admin)
New-NetFirewallRule -Name "PostgreSQL" -DisplayName "PostgreSQL Server" `
  -Direction Inbound -Protocol TCP -LocalPort 5432 -Action Allow

```

Windows Configuration Files

```
C:\Program Files\PostgreSQL\16\data\postgresql.conf
C:\Program Files\PostgreSQL\16\data\pg_hba.conf
C:\Program Files\PostgreSQL\16\data\pg_ident.conf
```

Docker Installation

Docker is ideal for development environments, CI/CD pipelines, and containerized deployments.

Basic Docker Run

```
# Pull the official PostgreSQL image
docker pull postgres:16

# Run a PostgreSQL container
docker run --name my-postgres \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_DB=myapp \
  -p 5432:5432 \
  -d postgres:16

```

```
# Verify container is running
docker ps

# Connect to PostgreSQL inside the container
docker exec -it my-postgres psql -U postgres

# Connect from the host using psql
psql -h localhost -p 5432 -U postgres -d myapp
```

Docker with Persistent Storage

```
# Create a named volume for data persistence
docker volume create pgdata

docker run --name my-postgres \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_DB=myapp \
  -p 5432:5432 \
  -v pgdata:/var/lib/postgresql/data \
  -d postgres:16

# Inspect volume
docker volume inspect pgdata
```

Docker Compose (Recommended for Development)

```
# docker-compose.yml
version: '3.8'

services:
  postgres:
    image: postgres:16
    container_name: postgres_dev
    restart: unless-stopped
    environment:
      POSTGRES_USER:      ${POSTGRES_USER:-postgres}
      POSTGRES_PASSWORD:  ${POSTGRES_PASSWORD:-devpassword}
      POSTGRES_DB:        ${POSTGRES_DB:-myapp}
      PGDATA:              /var/lib/postgresql/data/pgdata
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-scripts:/docker-entrypoint-initdb.d # Auto-run .sql files on init
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-postgres}"]
      interval: 10s
      timeout: 5s
```

```
    retries: 5

pgadmin:
  image: dpage/pgadmin4:latest
  container_name: pgadmin_dev
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@example.com
    PGADMIN_DEFAULT_PASSWORD: adminpassword
  ports:
    - "8080:80"
  depends_on:
    - postgres

volumes:
  postgres_data:
```

```
# Start services
docker-compose up -d

# View logs
docker-compose logs -f postgres

# Stop services
docker-compose down

# Stop and remove volumes (DESTROYS DATA)
docker-compose down -v

# Connect
docker-compose exec postgres psql -U postgres -d myapp
```

Docker Environment Variables

Variable	Description	Default
POSTGRES_PASSWORD	Superuser password (REQUIRED)	—
POSTGRES_USER	Superuser username	postgres
POSTGRES_DB	Default database name	Same as POSTGRES_USER
POSTGRES_INITDB_ARGS	Arguments to initdb	—
POSTGRES_HOST_AUTH_METHOD	Auth method for connections	scram-sha-256
PGDATA	Data directory path	/var/lib/postgresql/data

Kubernetes Installation

For production Kubernetes deployments, use the **CloudNativePG** operator — the most mature PostgreSQL operator for Kubernetes.

Method 1: CloudNativePG Operator

```
# Step 1: Install the CloudNativePG operator
kubectl apply --server-side -f \
  https://raw.githubusercontent.com/cloudnative-pg/cloudnative-pg/release-
  1.23/releases/cnpg-1.23.0.yaml

# Verify operator is running
kubectl get pods -n cnpg-system
```

```
# cluster.yaml – PostgreSQL cluster definition
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: postgres-cluster
  namespace: default
spec:
  instances: 3                                # 1 primary + 2 replicas

  postgresql:
    parameters:
      max_connections: "200"
      shared_buffers: "256MB"

  bootstrap:
    initdb:
      database: myapp
      owner: myapp_user
      secret:
        name: myapp-db-secret                # Contains username/password

  storage:
    size: 50Gi
    storageClass: fast-ssd                   # Your storage class

  backup:
    barmanObjectStore:
      destinationPath: s3://my-bucket/postgres-backups
      s3Credentials:
        accessKeyId:
          name: backup-credentials
          key: ACCESS_KEY_ID
        secretAccessKey:
          name: backup-credentials
          key: SECRET_ACCESS_KEY
    wal:
      compression: gzip
```

```
# Apply the cluster
kubectl apply -f cluster.yaml

# Watch cluster come up
kubectl get cluster postgres-cluster -w

# View pods
kubectl get pods -l cnpg.io/cluster=postgres-cluster

# Connect to primary
kubectl exec -it postgres-cluster-1 -- psql -U postgres
```

Method 2: Bitnami Helm Chart

```
# Step 1: Add Bitnami Helm repo
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Step 2: Install PostgreSQL
helm install my-postgres bitnami/postgresql \
  --set auth.postgresPassword=mysecretpassword \
  --set auth.database=myapp \
  --set primary.persistence.size=20Gi \
  --namespace postgres \
  --create-namespace

# Step 3: Check status
kubectl get pods -n postgres
helm status my-postgres -n postgres

# Step 4: Connect
kubectl run -it --rm --image postgres:16 psql-client --restart=Never -- \
  psql -h my-postgres-postgresql.postgres.svc.cluster.local -U postgres

# Uninstall
helm uninstall my-postgres -n postgres
```

Kubernetes Secrets for Credentials

```
# db-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: myapp-db-secret
type: kubernetes.io/basic-auth
stringData:
  username: myapp_user
  password: SuperSecurePassword123!
```

```
kubectl apply -f db-secret.yaml
# Or create directly:
kubectl create secret generic myapp-db-secret \
  --from-literal=username=myapp_user \
  --from-literal=password=SuperSecurePassword123!
```

Post-Installation Configuration

Security Hardening

```
# 1. Set superuser password (if not already done)
sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD
'VeryStrongPassword123!';"

# 2. Create application user (don't use postgres for apps)
sudo -u postgres psql <<EOF
CREATE USER app_user WITH
    PASSWORD 'app_password_here'
    NOSUPERUSER
    NOCREATEDB
    NOCREATEROLE
    LOGIN;
CREATE DATABASE appdb OWNER app_user;
GRANT CONNECT ON DATABASE appdb TO app_user;
EOF

# 3. Configure pg_hba.conf for authentication
# Location: /etc/postgresql/16/main/pg_hba.conf (Ubuntu)
#           /var/lib/postgresql/16/data/pg_hba.conf (RHEL)
```

pg_hba.conf — Authentication Configuration

#	TYPE	DATABASE	USER	ADDRESS	METHOD
# Local connections via Unix socket					
	local	all	postgres		peer
	local	all	all		scram-sha-256
# IPv4 local connections					
	host	all	all	127.0.0.1/32	scram-sha-256
# IPv6 local connections					
	host	all	all	::1/128	scram-sha-256
# Remote connections (replace with your network CIDR)					
	host	appdb	app_user	10.0.0.0/8	scram-sha-256

```
# NEVER use this in production:
# host      all              all              0.0.0.0/0        trust # DANGEROUS!
```

postgresql.conf — Key Settings

```
# Connection Settings
listen_addresses = 'localhost'          # '*' for all interfaces; 'localhost' for
local only
port = 5432
max_connections = 100                   # Lower for connection pooling setups

# Memory (adjust to your server RAM)
shared_buffers = 256MB                  # 25% of RAM (e.g., 1GB for 4GB RAM server)
effective_cache_size = 768MB            # 75% of RAM (helps planner; doesn't
allocate)
work_mem = 4MB                          # Per sort/hash operation per session
maintenance_work_mem = 64MB             # For VACUUM, CREATE INDEX

# WAL / Durability
wal_level = replica                     # For streaming replication
max_wal_senders = 10                    # Number of replication connections
synchronous_commit = on                 # Durability (off for higher write
throughput)

# Query Planner
random_page_cost = 1.1                  # Set to 1.1 for SSDs, 4.0 for HDDs
effective_io_concurrency = 200           # Set to 200 for SSDs, 2 for HDDs

# Logging
log_destination = 'stderr'
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_min_duration_statement = 1000        # Log queries taking > 1 second
log_line_prefix = '%t [%p]: [%l-1] '
log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits = on

# Autovacuum (never disable in production)
autovacuum = on
autovacuum_max_workers = 3
autovacuum_naptime = 1min

# Locale
datestyle = 'iso, mdy'
timezone = 'UTC'                        # Always use UTC for server timezone
```

After editing configuration files:

```
# Reload config (no restart needed for most settings)
sudo systemctl reload postgresql

# For settings that require restart:
sudo systemctl restart postgresql

# Or from within psql:
SELECT pg_reload_conf();
```

Verification Steps

```
# 1. Check PostgreSQL is running
sudo systemctl status postgresql
# Or:
pg_isready -h localhost -p 5432

# 2. Check version
psql --version
psql -U postgres -c "SELECT version();"

# 3. Check that you can connect
sudo -u postgres psql -c "SELECT 'Connected successfully!' AS status;"

# 4. Check listeners
ss -tlnp | grep 5432
# Or:
netstat -tlnp | grep 5432

# 5. Check configuration
sudo -u postgres psql -c "SHOW config_file;"
sudo -u postgres psql -c "SHOW hba_file;"
sudo -u postgres psql -c "SHOW data_directory;"

# 6. Check that databases are accessible
sudo -u postgres psql -l

# 7. Create a test database and table
sudo -u postgres psql <<EOF
CREATE DATABASE test_verify;
\c test_verify
CREATE TABLE test_table (id SERIAL, msg TEXT);
INSERT INTO test_table (msg) VALUES ('Installation successful!');
SELECT * FROM test_table;
DROP DATABASE test_verify;
EOF

echo "All verification steps passed!"
```

Troubleshooting

Issue 1: "could not connect to server: Connection refused"

```
# Check if PostgreSQL is running
sudo systemctl status postgresql

# Check if it's listening on the right port
ss -tlnp | grep 5432

# Check logs for errors
sudo journalctl -u postgresql -n 50 --no-pager

# Common causes:
# - Service not started: sudo systemctl start postgresql
# - Wrong port in client command
# - listen_addresses = 'localhost' but connecting remotely
#   Fix: Set listen_addresses = '*' in postgresql.conf
```

Issue 2: "FATAL: role 'username' does not exist"

```
# Create the role
sudo -u postgres psql -c "CREATE USER myuser WITH PASSWORD 'mypass';"

# Or, for psql to work without specifying username:
# The default user is your OS username; create a matching PostgreSQL role
sudo -u postgres psql -c "CREATE USER $(whoami) WITH SUPERUSER;"
psql postgres # Now works without -U flag
```

Issue 3: "FATAL: password authentication failed"

```
# Check pg_hba.conf authentication method for that connection type
cat /etc/postgresql/16/main/pg_hba.conf

# If using 'peer' auth for local connections, it checks OS username
# Switch to scram-sha-256 for password auth:
# Change: local all all peer
# To:      local all all scram-sha-256

# Reset the postgres user password
sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD 'newpassword';"

# Reload configuration
sudo systemctl reload postgresql
```

Issue 4: "FATAL: database 'username' does not exist"

```
# Connect to an existing database explicitly
psql -U postgres postgres
# Or:
psql -U postgres template1

# Then create your target database:
CREATE DATABASE mydb;
```

Issue 5: Port 5432 Already in Use

```
# Find what's using port 5432
sudo lsof -i :5432
sudo ss -tlnp | grep 5432

# Kill the conflicting process or change PostgreSQL's port:
# In postgresql.conf: port = 5433
# Then restart PostgreSQL
```

Issue 6: Permission Denied on Data Directory

```
# Fix ownership of data directory
sudo chown -R postgres:postgres /var/lib/postgresql/

# Fix permissions
sudo chmod 700 /var/lib/postgresql/16/main/
```

Issue 7: Docker Container Exits Immediately

```
# Check container logs
docker logs my-postgres

# Common cause: POSTGRES_PASSWORD not set
# Always set POSTGRES_PASSWORD environment variable

# Inspect container
docker inspect my-postgres
```

Issue 8: "could not translate host name"

```
# DNS resolution issue; use IP address instead of hostname
psql -h 127.0.0.1 -p 5432 -U postgres

# Or fix /etc/hosts
echo "127.0.0.1 localhost" >> /etc/hosts
```

ASCII Visual Diagrams

Diagram 1: Installation Path Decision Tree

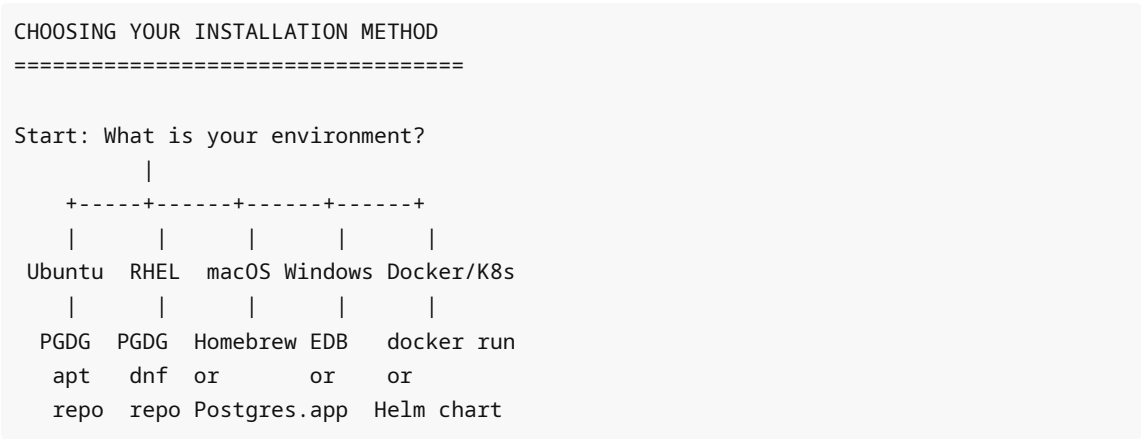


Diagram 2: PostgreSQL Directory Structure

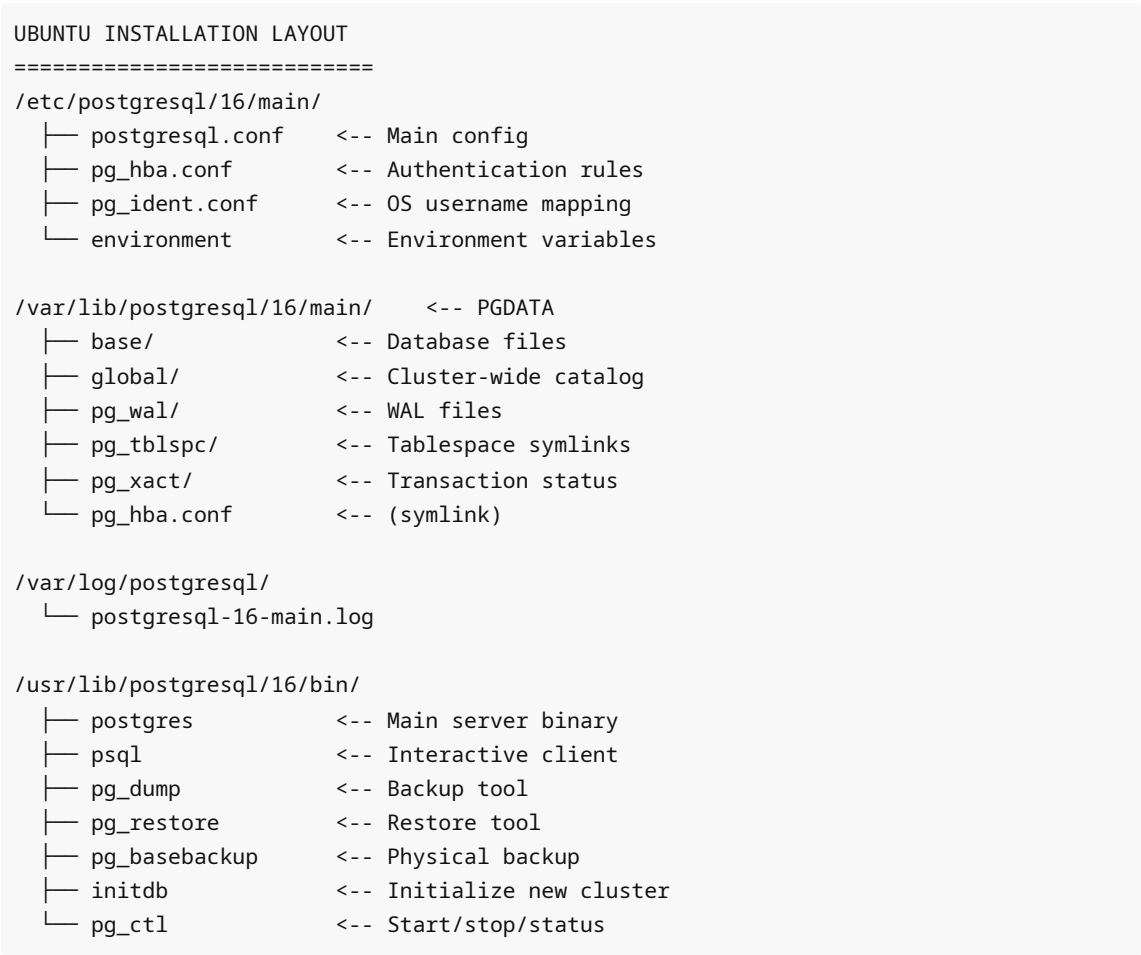


Diagram 3: Docker Architecture

HOST MACHINE

=====

```
+-----+
| docker run postgres:16 |
|                         |
| +-----Container-----+ |
| | PostgreSQL 16 Process | |
| | listening on port 5432 | |
| |                       | |
| | /var/lib/postgresql/data/ | |
| | (inside container)      | |
| +-----+-----+ |
| | Volume Mount          | |
| | v                     | |
| | /host/path/pgdata/    <-- Persists on host |
+-----+
```

PORT MAPPING:

host:5432 --> container:5432 (-p 5432:5432)

VOLUME MAPPING:

host:/data/pgdata --> container:/var/lib/postgresql/data

PostgreSQL Commands

```
# Cluster management
pg_ctl start -D /var/lib/pgsql/16/data/
pg_ctl stop -D /var/lib/pgsql/16/data/
pg_ctl status -D /var/lib/pgsql/16/data/
pg_ctl reload -D /var/lib/pgsql/16/data/

# Initialize a new cluster
initdb -D /new/data/directory --locale=en_US.UTF-8 --encoding=UTF8

# Backup
pg_dump -U postgres mydb > backup.sql
pg_dump -U postgres -Fc mydb > backup.dump # Custom format (compressed)

# Restore
psql -U postgres -d mydb < backup.sql
pg_restore -U postgres -d mydb backup.dump

# Check server is ready
pg_isready -h localhost -p 5432

# Show server configuration
postgres -C shared_buffers
```

SQL Examples

Example 1: First Commands After Installation

```
-- Check version
SELECT version();

-- Check current database
SELECT current_database();

-- List all databases
SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database;

-- Check configuration
SHOW all;

-- Check listening address and port
SHOW listen_addresses;
SHOW port;
```

Example 2: Create Production-Ready Setup

```
-- Create application database
CREATE DATABASE production_app
    ENCODING = 'UTF8'
    LC_COLLATE = 'en_US.UTF-8'
    LC_CTYPE = 'en_US.UTF-8'
    TEMPLATE = template0;

-- Create application user with limited privileges
CREATE USER app_service WITH
    PASSWORD 'SecureAppPassword!'
    NOSUPERUSER
    NOCREATEDB
    NOCREATEROLE
    LOGIN
    CONNECTION LIMIT 50;

-- Grant necessary permissions
GRANT CONNECT ON DATABASE production_app TO app_service;

\c production_app

GRANT USAGE ON SCHEMA public TO app_service;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_service;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO app_service;

-- Future objects also get permissions
```

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_service;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT USAGE, SELECT ON SEQUENCES TO app_service;
```

Example 3: Verify Installation Integrity

```
-- Check all critical settings
SELECT
    name,
    setting,
    unit,
    source
FROM pg_settings
WHERE name IN (
    'data_directory',
    'hba_file',
    'config_file',
    'listen_addresses',
    'port',
    'max_connections',
    'shared_buffers',
    'work_mem',
    'wal_level',
    'fsync',
    'synchronous_commit',
    'autovacuum',
    'timezone'
)
ORDER BY name;
```

Example 4: Post-Install Security Audit

```
-- Check for users with no password
SELECT username, passwd IS NULL AS no_password
FROM pg_shadow
WHERE passwd IS NULL AND username != 'replication';

-- Check for users with superuser privileges
SELECT username, usesuper, usecreatedb, usecreatorole
FROM pg_user
WHERE usesuper = TRUE;

-- Check databases with public access
SELECT datname, datacl FROM pg_database;

-- Check for unused roles
SELECT rolname FROM pg_roles
WHERE NOT rolcanlogin AND NOT rolsuper
```

```
AND NOT EXISTS (  
    SELECT 1 FROM pg_auth_members WHERE roleid = pg_roles.oid  
);
```

Example 5: Benchmark Installation

```
-- Quick write performance test  
CREATE TABLE bench_test (id SERIAL, data TEXT, val DOUBLE PRECISION);  
  
-- Insert 100,000 rows and time it  
\timing on  
  
INSERT INTO bench_test (data, val)  
SELECT md5(gs::TEXT), random()  
FROM generate_series(1, 100000) gs;  
  
-- Read performance  
SELECT COUNT(*), AVG(val), MAX(val) FROM bench_test;  
  
-- Cleanup  
DROP TABLE bench_test;  
\timing off
```

Example 6: Connection Test from Application

```
-- Test query for application health checks  
SELECT 1 AS connection_ok;  
  
-- More thorough check  
SELECT  
    1 AS status,  
    current_database() AS database,  
    current_user AS user,  
    inet_server_addr() AS server_addr,  
    inet_server_port() AS server_port,  
    now() AS server_time;
```

Example 7: Check Active Connections

```
-- View all current connections  
SELECT  
    pid,  
    username,  
    application_name,  
    client_addr,  
    backend_start,  
    state,  
    LEFT(query, 60) AS query
```

```
FROM pg_stat_activity
ORDER BY backend_start;
```

Example 8: pg_hba.conf Validation

```
-- Check current authentication configuration
SELECT type, database, user_name, address, auth_method
FROM pg_hba_file_rules
ORDER BY line_number;
```

Example 9: Enable and Verify Extensions

```
-- List available extensions
SELECT name, default_version, comment
FROM pg_available_extensions
ORDER BY name;

-- Install common extensions
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pg_stat_statements";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";
CREATE EXTENSION IF NOT EXISTS "unaccent";

-- Verify installed
SELECT extname, extversion FROM pg_extension ORDER BY extname;
```

Example 10: Log Analysis Query

```
-- After enabling pg_stat_statements, view top slow queries
SELECT
    ROUND(mean_exec_time::NUMERIC, 2) AS avg_ms,
    calls,
    ROUND(total_exec_time::NUMERIC, 2) AS total_ms,
    LEFT(query, 100) AS query_snippet
FROM pg_stat_statements
ORDER BY mean_exec_time DESC
LIMIT 20;
```

Common Mistakes

Mistake 1: Using `trust` Authentication in Production

```
# NEVER put this in pg_hba.conf for production:
host all all 0.0.0.0/0 trust # Anyone can connect as anyone!
```

```
# USE instead:
host all all 0.0.0.0/0 scram-sha-256
```

Mistake 2: Not Setting a Superuser Password

The `postgres` OS user and the `postgres` database role need strong passwords in production.

Mistake 3: Keeping `listen_addresses = 'localhost'` and Wondering Why Remote Connections Fail

```
# For remote access, set:
listen_addresses = '*'      # All interfaces
# Or specify interface IPs:
listen_addresses = '192.168.1.100'
```

Remember to also update `pg_hba.conf` to allow the remote host.

Mistake 4: Not Configuring Firewall Rules

Opening port 5432 in your application code but forgetting to allow it through the OS firewall (UFW, `firewalld`, Windows Firewall) is a common gotcha.

Mistake 5: Storing Data in the Default System Volume for Docker

```
# BAD: Data lost when container is removed
docker run -d postgres:16

# GOOD: Named volume persists data
docker run -d -v pgdata:/var/lib/postgresql/data postgres:16
```

Mistake 6: Not Backing Up Before Upgrading Major Versions

Major PostgreSQL upgrades (e.g., 15 → 16) require `pg_upgrade` or a dump-and-restore. Always take a full backup first.

Best Practices

1. **Use PGDG repository** for Linux — always gets the latest security patches.
 2. **Set strong passwords** for all database users before allowing remote access.
 3. **Use `scram-sha-256`** authentication — not `md5` (deprecated) or `trust`.
 4. **Configure `listen_addresses` carefully** — only expose to needed interfaces.
 5. **Mount a persistent volume** when running PostgreSQL in Docker.
 6. **Set `timezone = 'UTC'`** in `postgresql.conf` — avoids daylight-saving bugs.
 7. **Enable `pg_stat_statements`** from day one — invaluable for performance tuning.
 8. **Set `log_min_duration_statement = 1000`** to log slow queries.
 9. **Use Docker Compose** for local development — reproducible environments.
 10. **Use a managed service** (AWS RDS, Google Cloud SQL) for production unless you have DBA expertise.
-

Performance Considerations

```
# Initial tuning guidelines for a dedicated PostgreSQL server:
# (Replace 16GB with your actual RAM)

shared_buffers      = 4GB          # 25% of 16GB RAM
effective_cache_size = 12GB        # 75% of 16GB RAM
work_mem            = 64MB          # For heavy sorting; caution: multiplied by
connections
maintenance_work_mem = 1GB         # For VACUUM, CREATE INDEX
max_worker_processes = 8           # = CPU cores
max_parallel_workers = 8           # = CPU cores
max_parallel_workers_per_gather = 4 # Half of CPU cores
random_page_cost     = 1.1         # For SSD storage
effective_io_concurrency = 200      # For SSD storage
wal_buffers           = 64MB        # 3% of shared_buffers
checkpoint_completion_target = 0.9
default_statistics_target = 100
```

Interview Questions

1. How would you install PostgreSQL on Ubuntu for a production server?
2. What is the purpose of `pg_hba.conf` ?
3. What is the difference between `peer` and `scram-sha-256` authentication?
4. How do you make PostgreSQL accept remote connections?
5. How do you run PostgreSQL in Docker with persistent data?
6. What are the key configuration settings to tune after installation?
7. How would you deploy PostgreSQL on Kubernetes?
8. What should you do immediately after installing PostgreSQL for security?

Interview Answers

Q2: `pg_hba.conf`

`pg_hba.conf` (Host-Based Authentication) controls which clients can connect to which databases as which users, and how they authenticate. Each line specifies: connection type (local socket or TCP), database name, username, client IP address, and authentication method (`scram-sha-256`, `peer`, `md5`, `trust`, `cert`, etc.). PostgreSQL evaluates rules top-to-bottom and uses the first match.

Q3: `peer` vs `scram-sha-256`

`peer` authentication uses the OS to verify the connecting user's OS username matches the PostgreSQL username — no password needed. Only works for local Unix socket connections. `scram-sha-256` uses a password challenge-response protocol. SCRAM is the modern, secure authentication method for password-based auth (superseding the older `md5` method).

Q7: PostgreSQL on Kubernetes

The recommended approach is the **CloudNativePG operator** which manages PostgreSQL clusters as native Kubernetes resources. It handles automated failover, backup to S3/GCS, rolling upgrades, and connection

pooling. Alternatively, the Bitnami Helm chart provides a simpler deployment. Avoid running PostgreSQL in Kubernetes without an operator — manual StatefulSet management is error-prone.

Hands-on Exercises

Exercise 1: Fresh Installation

Install PostgreSQL on your platform (or use Docker). Connect, verify the version, and create a database called `training`.

Exercise 2: User and Permission Setup

Create three users: `admin_user` (superuser), `app_user` (CRUD access), and `report_user` (read-only). Set up appropriate permissions on the `training` database.

Exercise 3: Configuration Tuning

Edit `postgresql.conf` to set `log_min_duration_statement = 100` (log queries over 100ms). Run a slow query and verify it appears in the logs.

Exercise 4: Docker Compose

Write a `docker-compose.yml` that starts PostgreSQL and pgAdmin. Connect pgAdmin to the PostgreSQL instance.

Exercise 5: Security Audit

Run the security audit SQL queries from Example 4. Identify any security issues in your installation and fix them.

Solutions

Solution 1 (Docker Quick Start)

```
# Fastest way to get PostgreSQL running
docker run --name training-pg \
  -e POSTGRES_PASSWORD=trainingpass \
  -e POSTGRES_DB=training \
  -p 5432:5432 \
  -d postgres:16

docker exec -it training-pg psql -U postgres -d training -c "SELECT version();"
```

Solution 2

```
-- As superuser:
CREATE USER admin_user WITH SUPERUSER PASSWORD 'AdminPass!';
CREATE USER app_user WITH PASSWORD 'AppPass!' NOSUPERUSER NOCREATEDB
NOCREATEROLE;
CREATE USER report_user WITH PASSWORD 'ReportPass!' NOSUPERUSER NOCREATEDB
```

```
NOCREATEROLE;

\c training

-- App user: full CRUD
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO app_user;

-- Report user: read-only
GRANT USAGE ON SCHEMA public TO report_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO report_user;
```

Advanced Notes

pg_upgrade: Major Version Upgrade

```
# Upgrade from PostgreSQL 15 to 16
sudo pg_upgrade \
  --old-datadir=/var/lib/postgresql/15/main \
  --new-datadir=/var/lib/postgresql/16/main \
  --old-bindir=/usr/lib/postgresql/15/bin \
  --new-bindir=/usr/lib/postgresql/16/bin \
  --check # Dry run first!

# After verifying, run without --check
# Then run the generated update_extensions.sql
```

Logical Replication for Zero-Downtime Migration

For zero-downtime major version upgrades:

1. Set up logical replication from old version to new version
2. Let new version catch up
3. Switch application connection string
4. No downtime

Connection String URI Format

```
postgresql://[user[:password]@][host][:port][/dbname][?param1=value1]
```

Examples:

```
postgresql://app_user:password@localhost:5432/myapp
postgresql://app_user:password@db.example.com:5432/myapp?sslmode=require
postgresql://app_user@/var/run/postgresql/myapp (Unix socket)
```

Cross-References

- Previous: [08 - PostgreSQL Introduction](#)

- **Next:** [02 SQL Basics/01 ddl commands.md](#) — Start writing SQL
- **Related:** [08 - PostgreSQL Introduction](#) — Architecture overview
- **See Also:** [09 Administration/01 configuration.md](#) — Complete postgresql.conf guide
- **See Also:** [09 Administration/02 backup_restore.md](#) — Backup strategies
- **See Also:** [09 Administration/04 replication.md](#) — Setting up replication